

Java Support II — Interview Cheatsheet

Kaiser Permanente · Mindtel Global · Java · SQL · Kafka · Splunk · JIRA · **Print: 2 pages**

TRIAGE FRAMEWORK: DETECT → TRIAGE → MITIGATE → RESOLVE → DOCUMENT | **Bridge opener:** "Declaring Sev-2 for [service]. Impact: [X]. Mitigation: [Y]. Next update in 30 min."

5-DAY PLAN

- Day 1 Java: jstack, exceptions, thread dumps
- Day 2 SQL: validation, joins, locks, EXPLAIN
- Day 3 Kafka: lag, rebalance, poison pill
- Day 4 Splunk: SPL, correlationId, alerts
- Day 5 JIRA + 3 scenarios + STAR mock

1. JAVA — FIRST 5 MINUTES

#	Action	Look For
1	Blast radius	Service, env, region, % affected
2	Recent changes	Deploy, config, DB migration, cert
3	Health	/actuator/health, readiness vs liveness
4	Metrics	CPU, heap, threads, GC, error rate, p95
5	Splunk	ERROR spike, top exception, correlationId
6	Thread dump	3 dumps 10s apart if CPU/hung
7	Heap dump	Only if OOM — after thread dump

Commands

```
jstack <pid> | jcmd <pid> Thread.print
jstat -gcutil <pid> 1000 10
kubectl exec -it <pod> -- jstack 1
kubectl top pod -n <ns>
curl localhost:8080/actuator/health
curl ../metrics/hikaricp.connections.active
```

Thread Dump Scan Order

- 1. BLOCKED on same lock → deadlock/contention
- 2. RUNNABLE same stack frame → CPU hot loop
- 3. pool/http-nio WAITING on Hikari → pool exhausted
- 4. kafka-consumer stuck → slow DB write in handler

Top Exceptions

Exception	Cause	Action
OOM heap	Leak, large payload	Heap dump; -Xmx; Splunk payload
CannotGetJdbcConnection	Pool exhausted	Hikari metrics; blocking SQL
SQLTimeoutException	Slow query/lock	pg_stat_activity; EXPLAIN
SocketTimeoutException	Downstream slow	Trace dependency timeouts
RejectedExecutionException	Thread pool full	Queue depth; scale/shed load
Kafka TimeoutException	Broker/ISR issue	Lag; broker health

2. SQL — VALIDATION & TRIAGE

Workflow: Read-only → count → MAX(timestamp) → duplicates/orphans → locks → EXPLAIN

```
-- Row count
SELECT COUNT(*) FROM member_eligibility WHERE effective_date = CURRENT_DATE;
-- Duplicates
SELECT member_id, COUNT(*) FROM claims_staging GROUP BY 1 HAVING COUNT(*)>1;
-- Orphans
SELECT c.claim_id FROM claims c LEFT JOIN members m ON c.member_id=m.member_id
WHERE m.member_id IS NULL;
-- Stale feed
SELECT source_system, MAX(last_updated_ts) FROM eligibility_feed GROUP BY 1;
-- Blocking (PostgreSQL)
SELECT pid, NOW()-query_start AS dur, LEFT(query,120) FROM pg_stat_activity W
HERE state!='idle' ORDER BY dur DESC;
-- Latest plan per member (LATERAL)
SELECT m.member_id, e.plan_code FROM members m JOIN LATERAL (
  SELECT plan_code FROM member_eligibility e WHERE e.member_id=m.member_id
  ORDER BY effective_date DESC LIMIT 1) e ON TRUE;
```

3. KAFKA — LAG TRIAGE

30-sec answer: Producers → partitioned topics → consumer groups read offsets. Lag = log end – current offset.

```
kafka-consumer-groups.sh --bootstrap-server <broker>:9092 \
--group eligibility-consumer-prod --describe
kafka-topics.sh --bootstrap-server <broker>:9092 --describe --topic patient-e
ligibility-events
```

Signal	Cause	Action
Lag all partitions	Slow consumer	Thread dump; DB; scale ≤ partitions
Lag one partition	Hot key	Identify skewed member_id
Rebalance loop	> max.poll.interval	Faster handler; smaller batch
Poison pill	Bad message retries	DLT; skip; replay offset

Healthcare: No PHI in logs; partition key = member_id; lag = stale eligibility = member impact

4. SPLUNK — SPL TEMPLATES

Fields: index, sourcetype, level, correlationId, exception, host

```
-- Error spike
index=kp_apps level=ERROR earliest=-1h | timechart span=5m count
-- Top exceptions
index=kp_apps earliest=-30m | rex field=_raw "(?<ex>[a-zA-Z0-9_.]+Exception)"
| stats count by ex | sort - count
-- Trace request
index=kp_apps correlationId="<id>" earliest=-24h | sort _time | table _time h
ost level message
-- JDBC pool
index=kp_apps earliest=-2h ("CannotGetJdbcConnection" OR "HikariPool" OR "SQL
Timeout")
| stats count by host, message
-- Kafka issues
index=kp_apps earliest=-2h ("rebalance" OR "offset commit" OR "org.apache.kaf
ka")
| stats count by message
-- Deploy correlation
index=kp_deployments OR index=kp_apps earliest=-4h | sort _time
-- Alert
index=kp_apps level=ERROR earliest=-5m | stats count as c | where c > 50
```

5. JIRA / SEVERITY

Sev	Definition	Response
1	Patient care critical, widespread	Bridge, 15-min updates
2	Major degradation	War room, hourly updates
3	Limited impact	Business hours

Prioritize: Safety/compliance → patient impact → breadth → duration → workaround

JIRA comment: Impact | Status | Actions (Splunk/Kafka/SQL evidence) | Next steps | Links

Roles: Incident Commander (timeline) · SME (triage) · Scribe (JIRA) · No PHI in tickets

6. SCENARIO QUICK SCRIPTS

A: Kafka Eligibility Lag +45k

- 1. Sev-2; kafka-consumer-groups --describe
- 2. Splunk: SQLTimeout + lag timechart
- 3. Thread dump: Hikari WAITING; SQL blocking PID
- 4. Mitigate: kill blocker (runbook); pause batch; don't scale if at partition count

B: High-CPU Java (95%)

- 1. 3× jstack 10s apart — RUNNABLE same frame = hot loop
- 2. Splunk: trace correlationId to bad input
- 3. Mitigate: feature flag off; restart hot pods; engage dev with dumps

C: Wrong Eligibility Plan

- 1. SQL: member_eligibility vs enrollment source — FULL OUTER JOIN mismatch
- 2. Check feed MAX(timestamp); Kafka lag may be 0
- 3. Splunk: JsonMappingException = schema drift; replay from offset

7. STAR STORIES (YOUR AMDOCS)

Incident lead: USCC go-live Mexico — 15+ incidents, 30-min JIRA updates, Splunk correlation, 3 UAT cycles, on-time sign-off.

Prioritization: Kafka lag (broad) first → OOM to teammate → single-account SQL last with rationale in JIRA.

Coordination: XL Axiata — unified Splunk dashboard, 45 min → 5 min detection (89%), blameless RCA.

ELEVATOR PITCH: Hands-on Java production support — 4+ yrs Spring Boot microservices, Splunk triage, Kafka pipeline monitoring, SQL data validation, JIRA incident coordination including onsite go-live cutovers (5M+ txn/month).

DAY-BEFORE 30 MIN: Thread dump order · 5 SPL queries · kafka-consumer-groups --describe · SQL blocking query · 3 STAR stories · Detect→Triage→Mitigate→Resolve→Document